

UNIT I

BOOLEAN ALGEBRA AND LOGIC GATES

Number Systems - Arithmetic Operations - Binary Codes- Boolean Algebra and Logic Gates - Theorems and Properties of Boolean Algebra - Boolean Functions - Canonical and Standard Forms - Simplification of Boolean Functions using Karnaugh Map - Logic Gates – NAND and NOR Implementations.

Introduction

Basically there are two types of signals in electronics,

- i) Analog
- ii) Digital

Digital systems

Advantages:

- ❖ The usual advantages of digital circuits when compared to analog circuits are: Digital systems interface well with computers and are easy to control with software. New features can often be added to a digital system without changing hardware.
- ❖ Often this can be done outside of the factory by updating the product's software. So, the product's design errors can be corrected after the product is in a customer's hands.
- ❖ Information storage can be easier in digital systems than in analog ones. The noise-immunity of digital systems permits data to be stored and retrieved without degradation.
- ❖ In an analog system, noise from aging and wear degrade the information stored.
- ❖ In a digital system, as long as the total noise is below a certain level, the information can be recovered perfectly.

Disadvantages:

- ❖ In some cases, digital circuits use more energy than analog circuits to accomplish the same tasks, thus producing more heat as well. In portable or battery-powered systems this can limit use of digital systems.
- ❖ Digital circuits are sometimes more expensive, especially in small quantities. The sensed world is analog, and signals from this world are analog quantities.
- ❖ Digital circuits are sometimes more expensive, especially in small quantities. The sensed world is analog, and signals from this world are analog quantities.
- ❖ For example, light, temperature, sound, electrical conductivity, electric and magnetic fields are analog.



REVIEW OF NUMBER SYSTEMS

Many number systems are in use in digital technology. The most common are the decimal, binary, octal, and hexadecimal systems. The decimal system is clearly the most familiar to us because it is tools that we use every day.

Types of Number Systems are

- ❖ Decimal Number system
- ❖ Binary Number system
- ❖ Octal Number system
- ❖ Hexadecimal Number system

Table: Types of Number Systems

DECIMAL	BINARY	OCTAL	HEXADECIMAL
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Table: Number system and their Base value

Number Systems		
System	Base	Digits
Binary	2	0 1
Octal	8	0 1 2 3 4 5 6 7
Decimal	10	0 1 2 3 4 5 6 7 8 9
Hexadecimal	16	0 1 2 3 4 5 6 7 8 9 A B C D E F



Code Conversion:

- ❖ Converting from one code form to another code form is called code conversion, like converting from binary to decimal or converting from hexadecimal to decimal.

Binary-To-Decimal Conversion:

Any binary number can be converted to its decimal equivalent simply by summing together the weights of the various positions in the binary number which contain a 1.

Binary	Decimal
11011 ₂	
$=2^4+2^3+0+2^1+2^0$	$=16+8+0+2+1$
Result	27 ₁₀

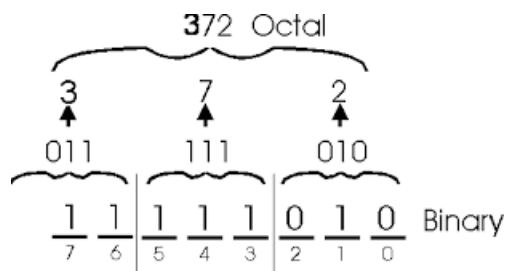
Decimal to binary Conversion:

Division	Remainder	Binary
25/2	=12+remainder of 1	1 (Least Significant Bit)
12/2	=6 +remainder of 0	0
6/2	=3 +remainder of 0	0
3/2	=1 +remainder of 1	1
½	=0 +remainder of 1	1 (Most Significant Bit)
Result	25 ₁₀	=11001 ₂

Binary to octal:

Example: 100 111010₂=(100)(111)(010)₂=4 7 2₈

Octal to Binary:

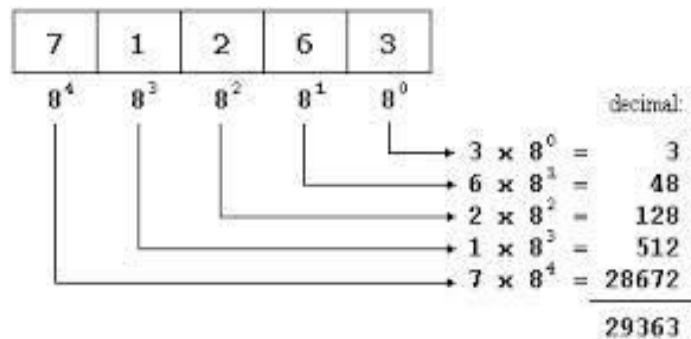


Decimal to octal:

Division	Result	Binary
177/8	=22+remainder of 1	1 (Least Significant Bit)
22/ 8	=2 +remainder of 6	6
2 / 8	=0 +remainder of 2	2 (Most Significant Bit)
Result	177 ₁₀	=261 ₈
Binary		=010110001 ₂

Octal to Decimal:

Example:



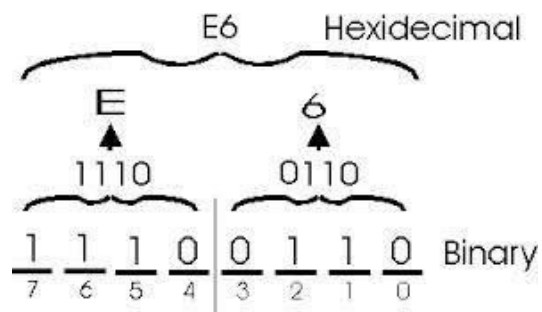
Decimal to Hexadecimal:

Division	Result	Hexadecimal
378/16	=23+remainder of 10	A(Least Significant Bit)23
23/16	=1 +remainder of 7	7
1/16	=0 +remainder of 1	1 (Most Significant Bit)
Result	37810	=17A ₁₆
Binary		=00010111 1010 ₂

Binary-To-Hexadecimal:

Example: $1011\ 0010\ 1111_2 = (1011)\ (0010)\ (1111)_2 = B2F_{16}$

Hexadecimal to binary:



Octal-To-Hexadecimal / Hexadecimal-To-Octal Conversion:

- ❖ Convert Octal (Hexadecimal) to Binary first.
- ❖ Regroup the binary number by three bits per group starting from LSB if Octal is required.
- ❖ Regroup the binary number by four bits per group starting from LSB if Hexadecimal is required.

Octal to Hexadecimal:

(May 2014)

Octal	Hexadecimal
=2 6 5 0	
= 010 110101000	=0101 1010 1000(Binary)
Result	=(5A8) ₁₆

Hexadecimal to octal:

Hexadecimal	Octal
(5A8) ₁₆	=0101 1010 1000(Binary)
	=010 110101000(Binary)
Result	=2 6 5 0(Octal)

1's and 2's complement:

- ❖ Complements are used in digital computers to simplify the subtraction operation and for logical manipulation.
- ❖ There are TWO types of complements for each base- r system: the radix complement and the diminished radix complement.
- ❖ The first is referred to as the r 's complement and the second as the $(r-1)$'s complement, when the value of the base r is substituted in the name. The two types are referred to as the 2's complement and 1's complement for binary numbers and the 10's complement and 9's complement for decimal numbers.

Note:

- The 1's complement of a binary number is the number that results when we change all 1's to zeros and the zeros to ones.
- The 2's complement is the binary number that results when we add 1 to the 1's complement.
- It is used to represent negative numbers.

2's complement = 1's complement + 1

Example 1) : Find 1's complement of $(1101)_2$

Sol: 1 1 0 1 ← Number
 0 0 1 0 ← 1's complement

Example 2) : Find 2's complement of

$(1001)_2$ Sol: 1 0 0 1 number

 0 1 1 0 ← 1's complement
 + 1
 ———
 0 1 1 1

Diminished Radix Complement:

Given a number N in base r having n digits, the $(r-1)$'s complement of N , i.e., its diminished radix complement, is defined as $(r^n - 1) - N$.

The 9's complement of 546700 is $999999 - 546700 =$

453299. The 9's complement of 012398 is $999999 -$

012398 = 987601.

Radix Complement:

The r 's complement of an n -digit number N in base r is defined as $r^n - N$ for $N \neq 0$ and as 0 for $N = 0$.

For examples:

The 10's complement of 012398 is 987602
The 10's complement of 246700 is 753300

Model 1:*(Dec 2009)*

Using 10's complement, subtract 72532-3250.

$$\begin{array}{r}
 M = 72532 \\
 10\text{'s complement of } N = +96750 \\
 \text{Sum} = 169282 \\
 \text{Discard end carry } 10^5 = -100000 \\
 \text{Answer} = 69282
 \end{array}$$

Model 2:

Using 10's complement, subtract 3250-72532.

$$\begin{array}{r}
 M = 03250 \\
 10\text{'s complement of } N = +27468 \\
 \text{Sum} = 30718
 \end{array}$$

Model 3:

Given the two binary numbers $X = 1010100$ and $Y = 1000011$, perform the subtraction (a) $X - Y$ and (b) $Y - X$ by using 2's complements. [NOV – 2019]

$$\begin{array}{r}
 \text{(a)} \quad X = 1010100 \\
 2\text{'s complement of } Y = + 0111101 \\
 \text{Sum} = 10010001 \\
 \text{Discard end carry } 2^7 = -10000000 \\
 \text{Answer: } X - Y = 0010001
 \end{array}$$

$$\text{(b)} \quad Y = 1000011$$

$$\begin{array}{r}
 2\text{'s complement of } X = 0101100 \\
 \text{Sum} = 1101111
 \end{array}$$

There is no end carry. Therefore, the answer is $Y - X = -(2\text{'s complement of } 1101111) = -0010001$.

Model 4:

Given the two binary numbers $X=1010100$ and $Y=1000011$, perform the subtraction (a) $X-Y$ and (b) $Y-X$ by using 1's complements. (Dec 2009)

(a) $X-Y = 1010100-1000011$

$$\begin{array}{r} X = 1010100 \\ 1's \text{ complement of } Y = \\ +0111100 \text{ Sum} = 10010000 \\ \text{End around carry} = +1 \\ \text{Answer: } X-Y = 0010001 \end{array}$$

(b) $Y-X = 1000011-1010100$

$$\begin{array}{r} Y = 1000011 \\ 1's \text{ complement of } X = +0101011 \\ \text{Sum} = 1101110 \end{array}$$

There is no end carry. Therefore, the answer is $Y-X = -(1's \text{ complement of } 1101110) = -0010001$.

ARITHMETIC OPERATIONS

Binary Addition:

Rules of Binary Addition

- $0+0=0$
- $0+1=1$
- $1+0=1$
- $1+1=0$, and carry 1 to the next most significant bit

Example:

Add: $00011010+00001100=00100110$

$$\begin{array}{r} 11 \\ 00011010 \\ +00001100 \\ \hline 00100110 \end{array}$$

Binary Subtraction:

Rules of Binary Subtraction

- $0 - 0 = 0$
- $0 - 1 = 1$, and borrow 1 from the next more significant bit
- $1 - 0 = 1$
- $1 - 1 = 0$

Example:

Sub: $00100101 - 00010001 = 00010100$

$$\begin{array}{r} 00100101 \\ - 00010001 \\ \hline 00010100 \end{array}$$

Binary Multiplication:

Rules of Binary Multiplication

- $0 \times 0 = 0$
- $0 \times 1 = 0$
- $1 \times 0 = 0$
- $1 \times 1 = 1$, and no carry or borrow bits

Example: Multiply the following binary numbers:

(a) 0111 and 110

(b) 1.011 and 10.01 .

(a) 0111×1101

				0	1	1	1	Multiplicand
				×	1	1	0	1
					0	1	1	1
			0	0	0	0		Partial Products
		0	1	1	1			
	0	1	1	1				
1	0	1	1	0	1	1		Final Product

(b) 1.011×10.01

				1.	0	1	1	Multiplicand
				×	1	0.	0	1
					1	0	1	1
			0	0	0	0	0	Partial Products
		0	0	0	0			
	1	0	1	1				
1	1	.	0	0	0	1	1	Final Product

Binary Division:

Binary division is the repeated process of subtraction, just as in decimal division.

Example: Divide the following

(a) $11001 \div 101$

$$\begin{array}{r} 101 \overline{) 11001} \\ \underline{110} \\ 00101 \\ \underline{0010} \\ 0011 \\ \underline{0010} \\ 0011 \\ \underline{0010} \\ 0001 \end{array}$$

(b) $11110 \div 1001$

$$\begin{array}{r} 1001 \overline{) 11110} \\ \underline{1001} \\ 01100 \\ \underline{0100} \\ 01001 \\ \underline{0100} \\ 00010 \\ \underline{0001} \\ 00000 \end{array}$$

BINARYCODES

Explain the various codes used in digital systems with an example.(or)Explain in detail about Binarycodes with an example

- In digital systems a variety of codes are used to serve different purposes, such as data entry, arithmeticoperation, error detection and correction, etc.
- Selection of a particular code depends on the requirement.
- Binarycodesarecodeswhicarerepresentedinbinarysystemwithmodification from the original ones.
- Codes can be broadly classified into five groups.
 - (i) Weighted Binary Codes
 - (ii) Non-weighted Codes
 - (iii) Error-detection Codes
 - (iv) Error-correcting Codes
 - (v) Alphanumeric Codes

Weighted Binary Codes

- If each position of a number represents a specific weight then the coding scheme is called weightedbinary code.

BCD Code or 8421 Code:

- The full form of BCD is 'Binary-Coded Decimal'. Since this is a coding scheme relating

decimal and binary numbers, *four bits are required* to code each decimal number.



- A decimal number in BCD (8421) is the same as its equivalent binary number only when the number is between 0 and 9. A BCD number greater than 10 looks different from its equivalent binary number, even though both contain 1's and 0's. Moreover, the binary combinations 1010 through 1111 are not used and have no meaning in BCD.
- Consider decimal 185 and its corresponding value in BCD and binary:

$$(185)_{10} = (0001\ 1000\ 0101)_{\text{BCD}} = (10111001)_2$$

- For example, $(35)_{10}$ is represented as 0011 0101 using BCD code, rather than $(100011)_2$
- *Example: Give the BCD equivalent for the decimal number 589.*

The decimal number is 5 8 9

BCD code is 0101 1000 1001

Hence, $(589)_{10} = (010110001001)_{\text{BCD}}$

2421 Code:

- Another weighted code is 2421 code. The weights assigned to the four digits are 2, 4, 2, and 1.
- The 2421 code is the same as that in BCD from 0 to 4. However, it varies from 5 to 9.
- For example, in this case the bit combination 0100 represents decimal 4; whereas the bit combination 1101 is interpreted as the decimal 7, as obtained from $2 \times 1 + 1 \times 4 + 0 \times 2 + 1 \times 1 = 7$.
- This is also a self-complementary code.

BCD Addition:

Examples:

- ❖ Consider the addition of $184 + 576 = 760$ in BCD:

BCD	1	1		
	0001	1000	0100	184
	+0101	0111	0110	+576
Binary sum	0111	10000	1010	
Add 6		0110	0110	
BCD sum	0111	0110	0000	760

- ❖ Add the following BCD numbers: (a) 1001 and 0100, (b) 00011001 and 00010100

Solution

(a)

1 0 0 1

+ 0 1 0 0

1 1 0 1

+ 0 1 1 0

0 0 0 1

1 0 0 1

1 0 0 1

1 0 0 1

1 0 0 1

1 0 0 1

1 0 0 1

1 0 0 1

1 0 0 1

1 0 0 1

1 0 0 1

1 0 0 1

1 0 0 1

→ Invalid BCD number

→ Add 6

→ Valid BCD number

(b)

0 0 0 1 1 0 0 1

+ 0 0 0 1 0 1 0 0

0 0 1 0 1 1 0 1

+ 0 1 1 0

0 0 1 1 0 0 1 1

0 0 1 1 0 0 1 1

0 0 1 1 0 0 1 1

0 0 1 1 0 0 1 1

0 0 1 1 0 0 1 1

0 0 1 1 0 0 1 1

0 0 1 1 0 0 1 1

→ Right group is invalid

→ Add 6

→ Valid BCD number

Four Different Binary Codes for the Decimal Digits

Decimal Digit	BCD 8421	2421	Excess-3	8, 4, -2, -1
0	0000	0000	0011	0000
1	0001	0001	0100	0111
2	0010	0010	0101	0110
3	0011	0011	0110	0101
4	0100	0100	0111	0100
5	0101	1011	1000	1011
6	0110	1100	1001	1010
7	0111	1101	1010	1001
8	1000	1110	1011	1000
9	1001	1111	1100	1111
	1010	0101	0000	0001
Unused	1011	0110	0001	0010
bit	1100	0111	0010	0011
combi-	1101	1000	1101	1100
nations	1110	1001	1110	1101
	1111	1010	1111	1110

Non-weighted Codes

- It basically means that each position of the binary number is not assigned a fixed value.
- Excess-3 codes and Gray codes are such non-weighted codes.

Excess-3 code:

- ❖ Excess-3 is an on-weighted code used to express decimal numbers. The code derives its name from the fact that each binary code is the corresponding 8421 code plus 0011(3).

Example: 1000 of 8421 (BCD)=1011 in Excess-3

❖ *Convert $(367)_{10}$ into its Excess-3 code.*

Solution. The decimal number is 3 6 7
Add 3 to each bit +3 +3 +3
Sum 6 9 10

Converting the above sum into 4-bit binary equivalent, we have a
4-bit binary equivalent of 0110 1001 1010

Hence, the Excess-3 code for $(367)_{10} = 0110 \ 1001 \ 1010$



Gray code:

- ❖ The gray code belongs to a class of codes called minimum change codes, in which only one bit in the code changes when moving from one code to the next.
- ❖ The Gray code is non-weighted code, as the position of bit does not contain any weight. In digital Gray code has got a special place.

Decimal Number	Binary Code	Gray Code
0	000 0	0000
1	000 1	0001
2	001 0	0011
3	001 1	0010
4	010 0	0110
5	010 1	0111
6	011 0	0101
7	011 1	0100
8	100 0	1100
9	100 1	1101
10	101 0	1111
11	101 1	1110
12	110 0	1010
13	110 1	1011
14	111 0	1001
15	111 1	1000

- ❖ The gray code is a reflective digital code which has the special property that any two subsequent numbers codes differ by only one bit. This is also called a *unit-distance code*.
- ❖ Important when an analog quantity must be converted to a digital representation. Only one bit changes between two successive integers which are being coded.

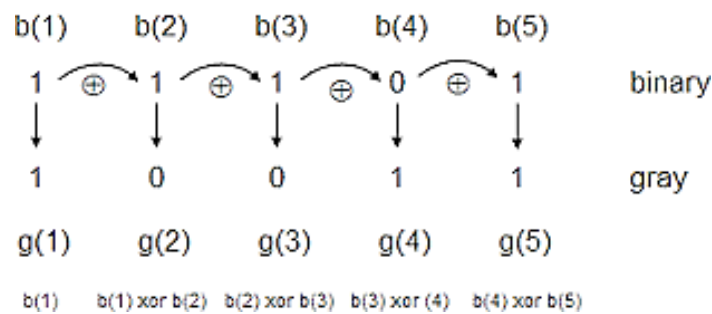
Example:**Binary to Gray Code Conversion:**

Any binary number can be converted into equivalent Gray code by the following steps:



- i) the MSB of the Gray code is the same as the MSB of the binary number;
 - ii) the second bit next to the MSB of the Gray code equals the Ex-OR of the MSB and second bit of the binary number; it will be 0 if there are same binary bits or it will be 1 for different binary bits;
 - iii) the third bit for Gray code equals the exclusive-OR of the second and third bits of the binary number, and similarly all the next lower order bits follow the same mechanism.
-

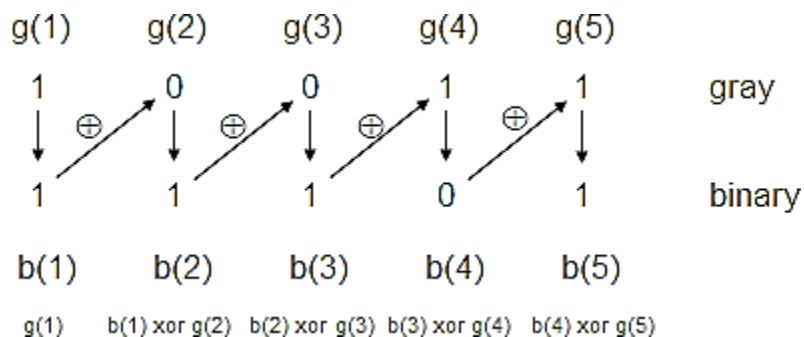




Gray Code to Binary Code Conversion:

Any Gray code can be converted into an equivalent binary number by the following steps:

- The MSB of the binary number is the same as the MSB of the Gray code.
- the second bit next to the MSB of the binary number equals the Ex-OR of the MSB of the binary number and second bit of the Gray code; it will be 0 if there are same binary bits or it will be 1 for different binary bits;
- the third bit for the binary number equals the exclusive-OR of the second bit of the binary number and third bit of the Gray code, and similarly all the next lower order bits follow the same mechanism.



Error detecting codes

- When data is transmitted from one point to another, like in wireless transmission, or it is just stored, like in hard disk and memories, there are chances that data may get corrupted.
- To detect these data errors, we use special codes, which are error detection codes.

Two types of parity

- **Even parity:** Checks if there is an even number of ones; if so, parity bit is zero. When the number of one's is odd then parity bit is set to 1.
- **Odd Parity:** Checks if there is an odd number of ones; if so, parity bit is zero. When the number of one's is even then parity bit is set to 1.

Error correcting code

- Error-correcting codes not only detect errors, but also correct them.
- This is used normally in Satellite communication, where turn-around delay is very high as is the

probability of data getting corrupt.

Hamming codes

- Hamming code adds a minimum number of bits to the data transmitted in a noisy channel, to be able to correct every possible one-bit error.
- It can detect (not correct) two-bit errors and cannot distinguish between 1-bit and 2-bit inconsistencies. It can't in general detect 3 (or more) bits errors.

Alphanumeric Codes

- An alphanumeric code is a binary code of a group of elements consisting of ten decimal digits, the 26 letters of the alphabet (both in uppercase and lowercase), and a certain number of special symbols such as #, /, &, %, etc.

ASCII (American Standard Code for Information Inter change)

- It is actually a 7-bit code, where a character is represented with seven bits.
- The character is stored as one byte with one bit remaining unused.
- But often the extra bit is used to extend the ASCII to represent an additional 128 characters.

EBCDIC codes

- EBCDIC stands for *Extended Binary Coded Decimal Inter change*.
- It is also an alphanumeric code generally used in IBM equipment and in large computers for communicating alphanumeric data.
- For the different alphanumeric characters the code grouping in this code is different from the ASCII code. It is actually an 8-bit code and a ninth bit is added as the parity bit.

Explain various theorems of Boolean algebra. (Nov – 2018) Definition:

Boolean algebra is an algebraic structure defined by a set of elements B , together with two binary operators. '+' and '-', provided that the following (Huntington) postulates are satisfied;

Theorems of Boolean algebra:

The theorems of Boolean algebra can be used to simplify many a complex Boolean expression and also to transform the given expression into a more useful and meaningful

T1: Commutative Law

- (a) $A + B = B + A$
- (b) $A B = B A$

T2: Associative Law

- (a) $(A + B) + C = A + (B + C)$
- (b) $(A B) C = A (B C)$

T3: Distributive Law

- (a) $A (B + C) = A B + A C$
- (b) $A + (B C) = (A + B) (A + C)$

T4: Identity Law

- (a) $A + A = A$
- (b) $A A = A$

T5: Negation Law

$$(\overline{\overline{A}}) = A \quad \text{and} \quad (\overline{\overline{A}}) = A$$

T6: Redundancy

- (a) $A + A B = A$
- (b) $A (A + B) = A$

T7: Operations with '0' & '1'

- (a) $0 + A = A$
- (b) $1 A = A$
- (c) $1 + A = 1$
- (d) $0 A = 0$

T8 : Complement laws

- (a) $\overline{\overline{A}} + A = 1$
- (b) $\overline{A} \cdot A = 0$

- T9 :**
- (a) $A + \overline{A} B = A + B$
 - (b) $A \cdot (\overline{A} + B) = A \cdot B$

Postulates of Boolean algebra:

The postulates of a mathematical system form the basic assumptions from which it is possible to deduce the rules, theorems, and properties of the system. The following are the important postulates of Boolean algebra:

1. $1 \cdot 1 = 1, 0 + 0 = 0$.
2. $1 \cdot 0 = 0 \cdot 1 = 0, 0 + 1 = 1 + 0 = 1$.
3. $0 \cdot 0 = 0, 1 + 1 = 1$
4. $1' = 0$ and $0' = 1$.

Many theorems of Boolean algebra are based on these postulates, which can be used to simplify Boolean expressions.

The operators and postulates have the following meanings:

- ✓ The binary operator + defines addition.
- ✓ The additive identity is 0.
- ✓ The additive inverse defines subtraction.
- ✓ The binary operator.(dot) defines multiplication.
- ✓ The multiplicative identity is 1.

✓ The only distributive law applicable is that of .(dot) over +:



$$a \cdot (b + c) = (a \cdot b) + (a \cdot c)$$

Two-Valued Boolean Algebra:

A two-valued Boolean algebra is defined on a set of two elements, $B = \{0, 1\}$, with rules for the two binary operators $+$ and $\cdot$ (dot) as shown in the following operator tables.

x	y	$x \cdot y$	x	y	$x + y$	x	x'
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

Duality Principle:

The *duality principle* states that every algebraic expression deducible from the postulates of Boolean algebra remains valid if the operators and identity elements are interchanged. If the *dual* of an algebraic expression is desired, we simply interchange OR and AND operators and replace 1's by 0's and 0's by 1's.

DeMorgan's theorem:

1. The complement of product is equal to the sum of their complements. $(X \cdot Y)' = X' + Y'$
2. The complement of sum is equal to the product of their complements. $(X + Y)' = X' \cdot Y'$

Basic Theorems:

State and prove postulates and theorems of Boolean algebra.

Postulates and Theorems of Boolean Algebra

Postulate 2	(a)	$x + 0 = x$	(b)	$x \cdot 1 = x$
Postulate 5	(a)	$x + x' = 1$	(b)	$x \cdot x' = 0$
Theorem 1	(a)	$x + x = x$	(b)	$x \cdot x = x$
Theorem 2	(a)	$x + 1 = 1$	(b)	$x \cdot 0 = 0$
Theorem 3, involution		$(x')' = x$		
Postulate 3, commutative	(a)	$x + y = y + x$	(b)	$xy = yx$
Theorem 4, associative	(a)	$x + (y + z) = (x + y) + z$	(b)	$x(yz) = (xy)z$
Postulate 4, distributive	(a)	$x(y + z) = xy + xz$	(b)	$x + yz = (x + y)(x + z)$
Theorem 5, DeMorgan	(a)	$(x + y)' = x'y'$	(b)	$(xy)' = x' + y'$
Theorem 6, absorption	(a)	$x + xy = x$	(b)	$x(x + y) = x$

THEOREM 1(a): $x + x = x$.

Statement	Justification
$x + x = (x + x) \cdot 1$	postulate 2(b)
$= (x + x)(x + x')$	5(a)
$= x + xx'$	4(b)
$= x + 0$	5(b)
$= x$	2(a)

THEOREM 1(b): $x \cdot x = x$.

Statement	Justification
$x \cdot x = xx + 0$	postulate 2(a)
$= xx + xx'$	5(b)
$= x(x + x')$	4(a)
$= x \cdot 1$	5(a)
$= x$	2(b)

THEOREM 2(a): $x + 1 = 1$.

Statement	Justification
$x + 1 = 1 \cdot (x + 1)$	postulate 2(b)
$= (x + x')(x + 1)$	5(a)
$= x + x' \cdot 1$	4(b)
$= x + x'$	2(b)
$= 1$	5(a)

THEOREM 2(b): $x \cdot 0 = 0$ by duality.

THEOREM 3: $(x')' = x$. From postulate 5, we have $x + x' = 1$ and $x \cdot x' = 0$, which together define the complement of x . The complement of x' is x and is also $(x')'$.

THEOREM 6(a): $x + xy = x$.

Statement	Justification
$x + xy = x \cdot 1 + xy$	postulate 2(b)
$= x(1 + y)$	4(a)
$= x(y + 1)$	3(a)
$= x \cdot 1$	2(a)
$= x$	2(b)

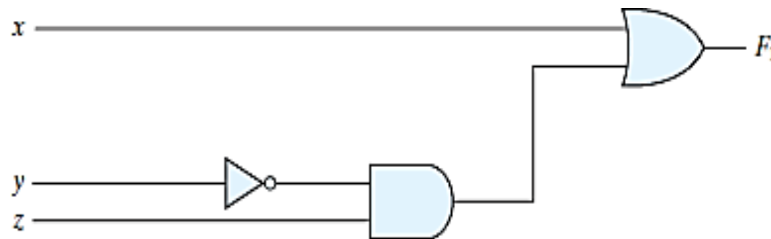
THEOREM 6(b): $x(x + y) = x$ by duality.

Boolean Functions

- ❖ Boolean algebra is an algebra that deals with binary variables and logic operations. A Boolean function described by an algebraic expression consists of binary variables, the constants 0 and 1, and the logic operation symbols.
- ❖ For a given value of the binary variables, the function can be equal to either 1 or 0.

Example, consider the Boolean function $F1 = x + y'z$

The function $F1$ is equal to 1 if x is equal to 1 or if both y and z are equal to 1. $F1$ is equal to 0 otherwise. The complement operation dictates that when $y' = 1$, $y = 0$. Therefore, $F1 = 1$ if $x = 1$ or if $y = 0$ and $z = 1$. A Boolean function expresses the logical relationship between binary variables and is evaluated by determining the binary value of the expression for all possible values of the variables. The gate implementation of $F1$ is shown below.



Example: Consensus Law: (function 4)

Simplify the following Boolean functions to a minimum number of literals.

1. $x(x' + y) = xx' + xy = 0 + xy = xy$.
2. $x + x'y = (x + x')(x + y) = 1(x + y) = x + y$.
3. $(x + y)(x + y') = x + xy + xy' + yy' = x(1 + y + y') = x$.
4. $xy + x'z + yz = xy + x'z + yz(x + x')$
 $= xy + x'z + xyz + x'yz$
 $= xy(1 + z) + x'z(1 + y)$
 $= xy + x'z$.
5. $(x + y)(x' + z)(y + z) = (x + y)(x' + z)$, by duality from function 4.

Complement of a function:

The complement of a function F is obtained from an interchange of 0's for 1's and 1's for 0's in the value of F .

Example:

1.

$$\begin{aligned}
 (A + B + C)' &= (A + x)' && \text{let } B + C = x \\
 &= A'x' && \text{by theorem 5(a) (DeMorgan)} \\
 &= A'(B + C)' && \text{substitute } B + C = x \\
 &= A'(B'C') && \text{by theorem 5(a) (DeMorgan)} \\
 &= A'B'C' && \text{by theorem 4(b) (associative)}
 \end{aligned}$$

2. Find the complement of the functions $F_1 = x'yz' + x'y'z$ and $F_2 = x(y'z' + yz)$.

By applying DeMorgan's theorems as many times as necessary, the complements are obtained as follows:

$$F_1' = (x'yz' + x'y'z)' = (x'yz')'(x'y'z)' = (x + y' + z)(x + y + z')$$

$$\begin{aligned} F_2' &= [x(y'z' + yz)]' = x' + (y'z' + yz)' = x' + (y'z')'(yz)' \\ &= x' + (y + z)(y' + z') \\ &= x' + yz' + y'z \end{aligned}$$

3. Find the complement of the functions $F_1 = x'yz' + x'y'z$ and $F_2 = x(y'z' + yz)$ by taking their duals and complementing each literals.

Solution:

1. $F_1 = x'yz' + x'y'z$.

The dual of F_1 is $(x' + y + z')(x' + y' + z)$.

Complement each literal: $(x + y' + z)(x + y + z') = F_1'$.

2. $F_2 = x(y'z' + yz)$.

The dual of F_2 is $x + (y' + z')(y + z)$.

Complement each literal: $x' + (y + z)(y' + z') = F_2'$.

Canonical and Standard forms:

Explain canonical SOP & POS form with suitable example.

- Binary logic values obtained by the logical functions and logic variables are in binary form. An arbitrary logic function can be expressed in the following forms.
 - (i) Sum of the Products (SOP)
 - (ii) Product of the Sums (POS)
- Boolean functions expressed as a sum of min terms or product of max terms are said to be in *canonical form*.

Product term:

The AND function is referred to as a product. The variable in a product term can appear either in complementary or uncomplimentary form. **Example:** ABC'

Sum term:

The OR function is referred to as a Sum. The variable in a sum term can appear either in complementary or uncomplimentary form. **Example:** $A+B+C'$

Sum of Product (SOP):

The logical sum of two or more logical product terms is called sum of product expression. It is basically an OR operation of AND operated variables. **Example:** $Y=AB+BC+CA$

Product of Sum (POS):

The logical product of two or more logical sum terms is called product of sum expression. It is basically an AND operation of OR operated variables. **Example:** $Y=(A+B).(B+C).(C+A)$

Minterm:

A product term containing all the K variables of the function in either complementary or uncomplimentary form is called Minterm or standard product.

Maxterm:

A sum term containing all the K variables of the function in either complementary or uncomplimentary form is called Maxterm or standard sum.

Minterms and Maxterms for Three Binary Variables

x	y	z	Minterms		Maxterms	
			Term	Designation	Term	Designation
0	0	0	$x'y'z'$	m_0	$x + y + z$	M_0
0	0	1	$x'y'z$	m_1	$x + y + z'$	M_1
0	1	0	$x'yz'$	m_2	$x + y' + z$	M_2
0	1	1	$x'yz$	m_3	$x + y' + z'$	M_3
1	0	0	$xy'z'$	m_4	$x' + y + z$	M_4
1	0	1	$xy'z$	m_5	$x' + y + z'$	M_5
1	1	0	xyz'	m_6	$x' + y' + z$	M_6
1	1	1	xyz	m_7	$x' + y' + z'$	M_7

Canonical SOP Expression:

The minterms whose sum defines the Boolean function are those which give the 1's of the function in a truth table.

Procedure for obtaining Canonical SOP expression:

- ✓ Examine each term in a given logic function. Retain if it is a minterm, continue to examine the next term in the same manner.
- ✓ Check for the variables that are missing in each product which is not minterm. Multiply the product by $(X+X')$, for each variable X that is missing.
- ✓ Multiply all the products and omit the redundant terms.

Example:

Express the Boolean function $F = A + B'C$ as a sum of minterms. (May -10)(Nov – 2018)

Solution:

The function has three variables: A, B, and C.

The first term A is missing two variables; therefore,

$$A = A(B + B') = AB + AB'$$

This function is still missing one variable, so

$$\begin{aligned} A &= AB(C + C') + AB'(C + C') \\ &= ABC + ABC' + AB'C + AB'C' \end{aligned}$$

The second term $B'C$ is missing one variable; hence,

$$B'C = B'C(A + A') = AB'C + A'B'C$$

Combining all terms, we have

$$F = A + B'C = ABC + ABC' + AB'C + AB'C' + A'B'C$$

But $AB'C$ appears twice, and according to theorem 1 ($x + x = x$), it is possible to remove one of those occurrences. Rearranging the minterms in ascending order, we finally obtain

$$F = A'B'C + AB'C' + AB'C + ABC' + ABC = m_1 + m_4 + m_5 + m_6 + m_7$$
$$m_7 F(A, B, C) = \sum(1, 4, 5, 6, 7)$$

Example: Obtain the canonical sum of product form of the following function. (May 2014)

$$F(A, B, C) = A + BC$$

$$= A(B + B')(C + C') + BC(A + A')$$

$$= (AB + AB')(C + C') + ABC + A'BC$$

$$= ABC + AB'C + ABC' + AB'C' + ABC + A'BC$$

$$= ABC + AB'C + ABC' + AB'C' + A'BC \text{ (as } ABC + ABC = ABC)$$

Hence the canonical sum of the product expression of the given function is

$$F(A, B, C) = ABC + AB'C + ABC' + AB'C' + A'BC$$

Canonical POS Expression:

The Maxterms whose product defines the Boolean function are those which give the 1's of the function in a truth table.

Procedure for obtaining Canonical POS expression:

- ✓ Examine each term in a given logic function. Retain if it is a maxterm, continue to examine the next term in the same manner.
- ✓ Check for the variables that are missing in each sum which is not maxterm. Add $(X.X')$, for each variable X that is missing.
- ✓ Expand the expression using distributive property eliminate the redundant terms.

Example:

Express the Boolean function $F = xy + x'z$ as a product of maxterms. First, convert the function into OR terms by using the distributive law:

$$\begin{aligned} F &= xy + x'z = (xy + x')(xy + z) \\ &= (x + x')(y + x')(x + z)(y + z) \\ &= (x' + y)(x + z)(y + z) \end{aligned}$$

The function has three variables: x , y , and z . Each OR term is missing one variable; therefore,

$$\begin{aligned} x' + y &= x' + y + zz' = (x' + y + z)(x' + y + z') \\ x + z &= x + z + yy' = (x + y + z)(x + y' + z) \\ y + z &= y + z + xx' = (x + y + z)(x' + y + z) \end{aligned}$$

Combining all the terms and removing those which appear more than once, we finally obtain

$$\begin{aligned} F &= (x + y + z)(x + y' + z)(x' + y + z)(x' + y + z') \\ &= M_0 M_2 M_4 M_5 \end{aligned}$$

A convenient way to express this function is as follows:

$$F(x, y, z) = \Pi(0, 2, 4, 5)$$

Example:

Obtain the canonical product of the sum form of the following function.

$$F(A, B, C) = (A + B')(B + C)(A + C') \quad (\text{Dec 2012})$$

Solution:

$$\begin{aligned} F(A, B, C) &= (A + B')(B + C)(A + C') \\ &= (A + B' + 0)(B + C + 0)(A + C' + 0) \\ &= (A + B' + CC')(B + C + AA')(A + C' + BB') \\ &= (A + B' + C)(A + B' + C')(A + B + C)(A' + B + C)(A + B + C') \\ &\quad (A + B' + C') \\ &\quad [\text{using the distributive property, as } X + YZ = (X + Y)(X + Z)] \\ &= (A + B' + C)(A + B' + C')(A + B + C)(A' + B + C)(A + B + C') \\ &\quad [\text{as } (A + B' + C')(A + B' + C) = A + B' + C'] \end{aligned}$$

Hence the canonical product of the sum expression for the given function is

$$F(A, B, C) = (A + B' + C)(A + B' + C')(A + B + C)(A' + B + C)(A + B + C')$$

Karnaugh Map (K-map):

- ❖ Using Boolean algebra to simplify Boolean expressions can be difficult. The Karnaugh map provides a simple and straight-forward method of minimizing Boolean expressions which represent combinational logic circuits.
- ❖ A Karnaugh map is a pictorial method of grouping together expressions with common factors and then eliminating unwanted variables.
- ❖ A Karnaugh map is a two-dimensional truth-table. Note that the squares are numbered so that the binary representations for the numbers of two adjacent squares differ in exactly one position.

Rules for Grouping together adjacent cells containing 1's:

- Groups must contain 1, 2, 4, 8, 16 (2^n) cells.
- Groups must contain only 1 (and X if don't care is allowed).
- Groups may be horizontal or vertical, but not diagonal.
- Groups should be as large as possible.
- Each cell containing a 1 must be in at least one group.
- Groups may overlap.
- Groups may wrap around the table. The leftmost cell in a row may be grouped with the rightmost cell and the top cell in a column may be grouped with the bottom cell.
- There should be as few groups as possible.

Obtaining Product Terms

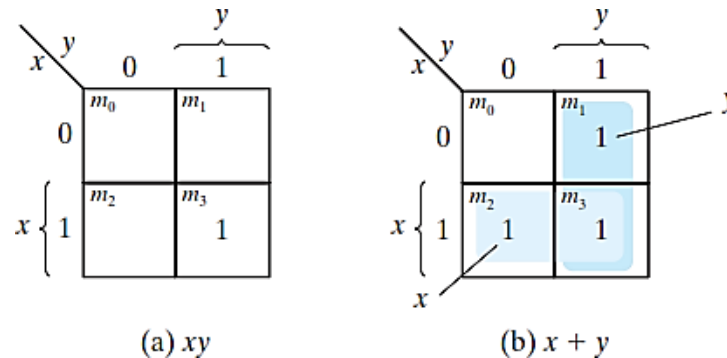
- If A is a variable that has value 0 in all of the squares in the grouping, then the complemented form A is in the product term.
- If A is a variable that has value 1 in all of the squares in the grouping, then the true form A is in the product term.



- If A is a variable that has value 0 for some squares in the grouping and value 1 for others, then it is not in the product term

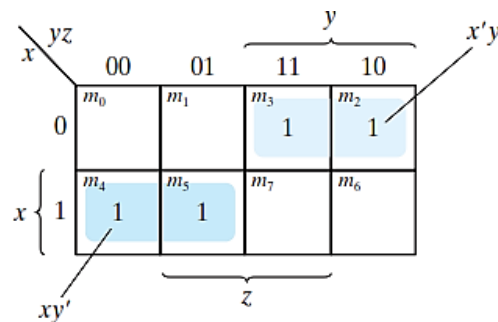
The Format of K-Maps:

K-Maps of 2 Variables:



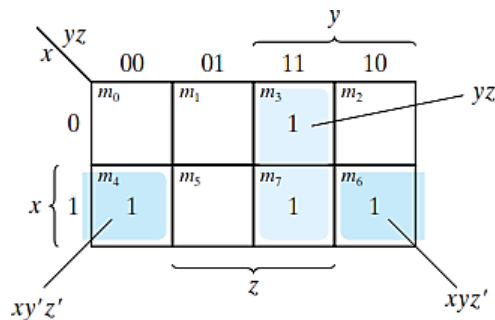
K-Maps of 3 Variables:

- ❖ Simplify the boolean function $F(x, y, z) = \sum(2, 3, 4, 5)$



$$F(x, y, z) = \sum(2, 3, 4, 5) = x'y + xy'$$

- ❖ Simplify the boolean function $F(x, y, z) = \sum 3, 4, 6, 7$



$$\text{Note: } xy'z' + xyz' = xz'$$

$$F(x, y, z) = \sum(3, 4, 6, 7) = yz + xz'$$

K-Maps of 4 Variables:

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6
m_{12}	m_{13}	m_{15}	m_{14}
m_8	m_9	m_{11}	m_{10}

(a)

		y			
		00	01	11	10
wx	yz				
	00	m_0 $w'x'y'z'$	m_1 $w'x'y'z$	m_3 $w'x'yz$	m_2 $w'x'yz'$
	01	m_4 $w'xy'z'$	m_5 $w'xy'z$	m_7 $w'xyz$	m_6 $w'xyz'$
	11	m_{12} $wxy'z'$	m_{13} $wxy'z$	m_{15} $wxyz$	m_{14} $wxyz'$
	10	m_8 $wx'y'z'$	m_9 $wx'y'z$	m_{11} $wx'yz$	m_{10} $wx'yz'$

(b)

- ❖ Simplify the boolean function $F(w,x,y,z) = \Sigma(0,1,2,4,5,6,8,9,12,13,14)$

		y			
		00	01	11	10
wx	yz				
	00	m_0 1	m_1 1		m_2 1
	01	m_4 1	m_5 1		m_6 1
	11	m_{12} 1	m_{13} 1		m_{14} 1
	10	m_8 1	m_9 1	m_{11}	m_{10}

Note: $w'y'z' + w'yz' = w'z'$

$xy'z' + xyz' = xz'$

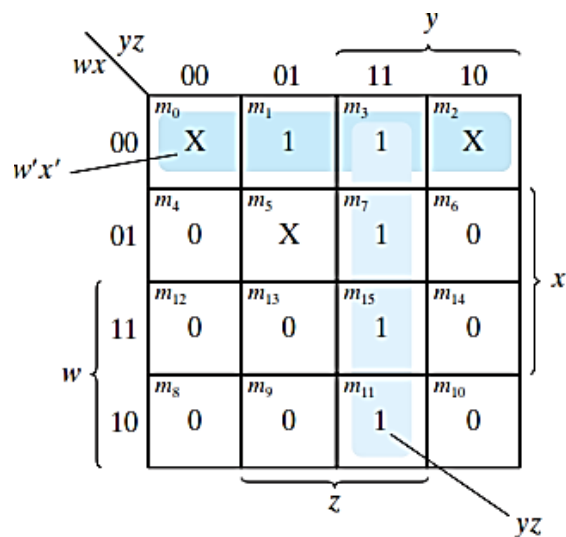
$$F(w, x, y, z) = \Sigma(0,1,2,4,5,6,8,9,12,13,14) = y' + w'z' + xz'$$

- ❖ Simplify the Boolean function

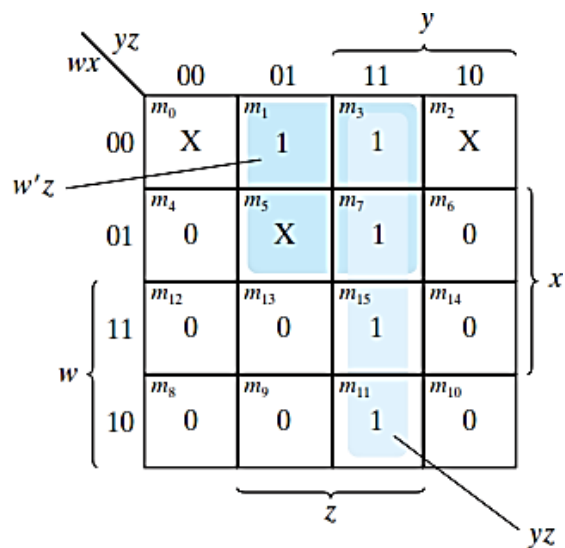
$$F(w, x, y, z) = \Sigma(1, 3, 7, 11, 15)$$

which has the don't-care conditions

$$d(w, x, y, z) = \Sigma(0, 2, 5)$$



(a) $F = yz + w'x'$



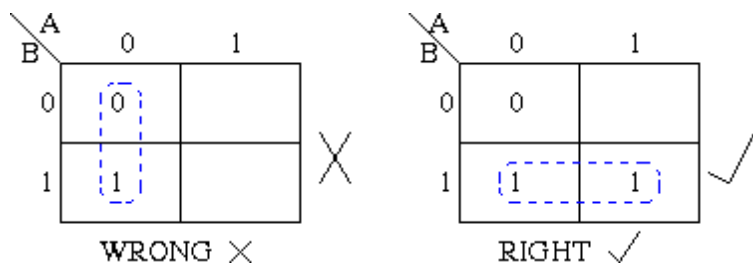
(b) $F = yz + w'z$

Note:

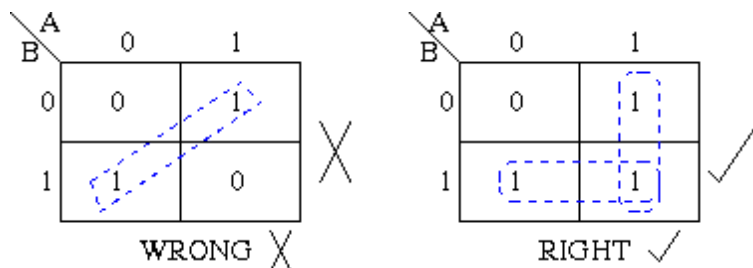
Karnaugh Maps - Rules of Simplification

The Karnaugh map uses the following rules for the simplification of expressions by *grouping* together adjacent cells containing *ones*

- Groups may not include any cell containing a zero



- Groups may be horizontal or vertical, but not diagonal.



- Groups must contain 1, 2, 4, 8, or in general 2^n cells. That is if $n = 1$, a group will contain two 1's since $2^1 = 2$. If $n = 2$, a group will contain four 1's since $2^2 = 4$.

$\backslash AB$	0	1
B		
0	1	1
1	0	0

Group of 2

RIGHT ✓

$\backslash AB$	00	01	11	10
C				
0	0	1	1	1
1	0	0	0	0

Group of 3

WRONG ✗

$\backslash AB$	0	1
B		
0	1	1
1	1	1

Group of 4

RIGHT ✓

$\backslash AB$	00	01	11	10
C				
0	1	1	1	1
1	0	0	0	1

Group of 5

WRONG ✗

- Each group should be as large as possible.

$\backslash AB$	00	01	11	10
C				
0	1	1	1	1
1	0	0	1	1

RIGHT ✓

$\backslash AB$	00	01	11	10
C				
0	1	1	1	1
1	0	0	1	1

WRONG ✗

(Note that no Boolean laws broken, but not sufficiently minimal)

- Each cell containing a *one* must be in at least one group.

$\backslash AB$	00	01	11	10
C				
0	0	0	1	1
1	0	0	0	1

Group I

Group II

1 present in at least one group.

- Groups may overlap.

AB		00	01	11	10
C	0	1	1	1	1
	1	0	0	1	1

Groups overlapping. ✓

RIGHT ✓

AB		00	01	11	10
C	0	1	1	1	1
	1	0	0	1	1

Groups not overlapping. ✗

WRONG ✗

- There should be as few groups as possible, as long as this does not contradict any of the previous rules.

AB		00	01	11	10
C	0	1	1	1	1
	1	0	0	1	1

RIGHT ✓

AB		00	01	11	10
C	0	1	1	1	1
	1	0	0	1	1

WRONG ✗

Summary:

1. No zeros allowed.
2. No diagonals.
3. Only power of 2 numbers of cells in each group.
4. Groups should be as large as possible.
5. Everyone must be in at least one group.
6. Overlapping allowed.
7. Wrap around allowed.
8. Fewest numbers of groups possible.

Don't care combination:

In certain digital systems, some input combinations never occur during the process of normal operation because those input conditions are guaranteed never to occur. Such input combinations are don't care conditions.

Completely specified functions:

If a function is completely specified, it assumes the value 1 for some input combinations and the value 0 for others.

Incompletely specified functions:

There are functions which assume the value 1 for some combinations and 0 for some other and either 0 or 1 for the remaining combinations. Such a functions are called incompletely specified .

Prime Implicants:

A prime implicant is a product term obtained by combining the maximum possible number of adjacent squares in the map. If a minterm in a square is covered by only one prime implicant, that prime implicant is said to be essential.

Quine - McCluskey (or) Tabulation Method

Minimization of Logic functions:

Steps:

- ✓ A set of all prime implicants of the function must be obtained.
- ✓ From the set of prime implicants, a set of essential implicants must be determined by preparing a prime implicant chart.
- ✓ The minterm which are not covered by the essential implicants are taken into consideration and a minimum cover is obtained from the remaining prime implicants.

Example:

(Nov-06,07,10,May- 10,08)

Simplify the boolean function $F(A,B,C,D) = \sum m (1,3,6,7,8,9,10,12,14,15) + \sum d (11,13)$ using QuineMcClusky method.

(Apr 2017)

Step:1

Minterms	Binary representation	Minterms	Binary representation
m ₁	0 0 0 1	m ₁	0 0 0 1 ✓
m ₃	0 0 1 1	m ₈	1 0 0 0 ✓
m ₆	0 1 1 0	m ₃	0 0 1 1 ✓
m ₇	0 1 1 1	m ₆	0 1 1 0 ✓
m ₈	1 0 0 0	m ₉	1 0 0 1 ✓
m ₉	1 0 0 1	m ₁₀	1 0 1 0 ✓
m ₁₀	1 0 1 0	m ₁₂	1 1 0 0 ✓
m ₁₂	1 1 0 0	m ₇	0 1 1 1 ✓
m ₁₄	1 1 1 0	m ₁₄	1 1 1 0 ✓
m ₁₅	1 1 1 1	dm ₁₁	1 0 1 1 ✓
dm ₁₁	1 0 1 1	dm ₁₃	1 1 0 1 ✓
dm ₁₃	1 1 0 1	m ₁₅	1 1 1 1 ✓

Step:2

Minterms	Binary representation	Minterms	Binary representation
1, 3	0 0 - 1 ✓	1, 3, 9, 11	- 0 - 1
1, 9	- 0 0 1 ✓	8, 9, 10, 11 ✓	1 0 - -
8, 9	1 0 0 - ✓	8, 10, 12, 14	1 - - 0
8, 10	1 0 - 0 ✓		
8, 12	1 - 0 0 ✓	6, 7, 14, 15 ✓	- 1 1 -
3, 7	0 - 1 1 ✓		
3, 11	- 0 1 1 ✓	12, 13, 14, 15	1 1 - -
6, 7	0 1 1 - ✓		
6, 14	- 1 1 0 ✓		
9, 11	1 0 - 1 ✓		
9, 13	1 - 0 1 ✓		
10, 14	1 - 1 0 ✓		
10, 11	1 0 1 - ✓		
12, 14	1 1 - 0 ✓		
12, 13	1 1 0 - ✓		
7, 15	- 1 1 1 ✓		
14, 15	1 1 1 - ✓		

Step:3

Prime implicants	Binary representation
1, 3, 9, 11 ($\bar{B} D$)	– 0 – 1
8, 9, 10, 11, 12, 13, 14, 15 (A)	1 – – –
6, 7, 14, 15 (BC)	– 1 1 –

Step:4

Prime implicants	m_1	m_3	m_6	m_7	m_8	m_9	m_{10}	m_{12}	m_{14}	m_{15}	dm_{11}	dm_{13}
1, 3, 9, 11 ($\bar{B} D$)	⊙	⊙				⊙					⊙	
8, 9, 10, 11, 12, 13, 14, 15					⊙	⊙	⊙	⊙	⊙	⊙	⊙	⊙
6, 7, 14, 15 (BC)			⊙	⊙					⊙	⊙		

$$\therefore F(A, B, C, D) = \bar{B}D + A + BC$$

***** Logic

.....

gates

Explain about different types of logic gates.(OR) What are Universal gates? Construct any four basic gates using only NOR gates and using only NAND gates. (May 2011)[NOV – 2019]

- ❖ A **logic gate** is an idealized or physical device implementing a Boolean function; that is, it performs a logical operation on one or more logical inputs, and produces a single logical output.

Positive and Negative Logic

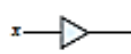
- ❖ The binary variables have two states, i.e. the logic '0' state or the logic '1' state. These logic states in digital systems such as computers.
- ❖ These are represented by two different voltage levels or two different current levels.
- ❖ If the more positive of the two voltage or current levels represents a logic '1' and the less positive of the two levels represents a logic '0', then the logic system is referred to as a **positive logic system**.
- ❖ If the more positive of the two voltage or current levels represents a logic '0' and the less positive of the two levels represents a logic '1', then the logic system is referred to as a **negative logic system**.

negative logic system.



Truth Table

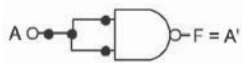
A truth table lists all possible combinations of input binary variables and the corresponding outputs of a logic system.

Name	Graphic symbol	Algebraic function	Truth table															
AND		$F = x \cdot y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$F = x + y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	1
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
Inverter		$F = x'$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	x	F	0	1	1	0									
x	F																	
0	1																	
1	0																	
Buffer		$F = x$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	x	F	0	0	1	1									
x	F																	
0	0																	
1	1																	
NAND		$F = (xy)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	1	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$F = (x + y)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	0
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
Exclusive-OR (XOR)		$F = xy' + x'y$ $= x \oplus y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
Exclusive-NOR or equivalence		$F = xy + x'y'$ $= (x \oplus y)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	1																

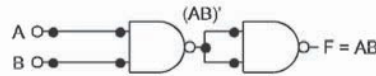
Universal Gates

- ❖ The OR, AND and NOT gates are the three basic logic gates as they together can be used to construct the logic circuit for any given Boolean expression.
- ❖ The NOR and NAND gates have the property that they individually can be used to hardware-implement a logic circuit corresponding to any given Boolean expression.
- ❖ That is, it is possible to use either only NAND gates or only NOR gates to implement any Boolean expression. This is so because a combination of NAND gates or a combination of NOR gates can be used to perform functions of any of the basic logic gates. It is for this reason that NAND and NOR gates are universal gates.

NAND gates and NOR gates are called *universal gates* or *universal building blocks*, as any type of gates or logic functions can be implemented by these gates. Figures Symbol show various logic function can be realized by NAND gates and Figures Symbol show there a lization of various logic gates by NOR gates.



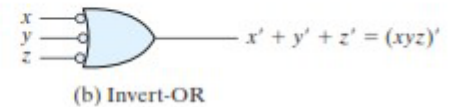
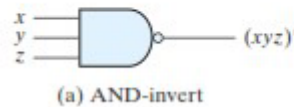
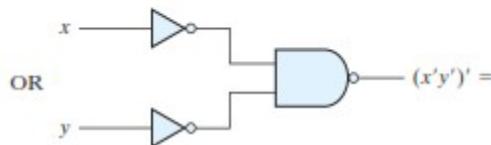
NOT function: $F = A'$ AND function: $F = AB$



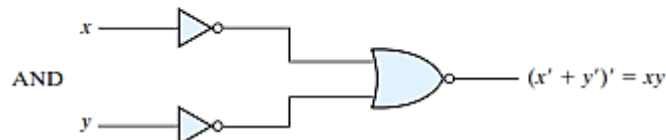
Implementation of basic gates using NAND gate: (convert AND gate to NAND gate)



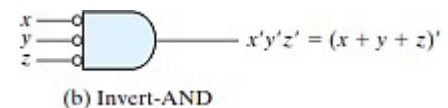
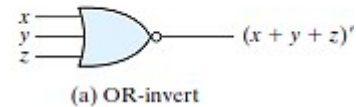
Logic operations with NAND gates



Implementation of basic gates using NOR gate: (convert OR gate to NOR gate)



Logic operations with NOR gates

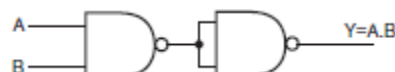


Implementation of basic gates using NAND gate:

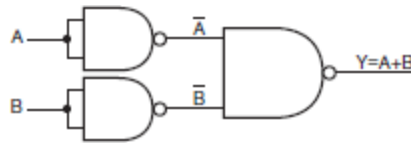
Inverter (NOT gate):



AND gate:



OR gate:

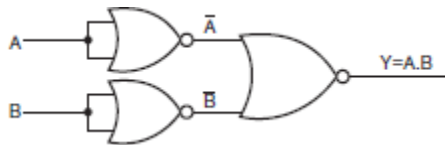


Implementation of basic gates using NOR gate:

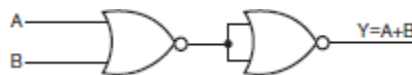
Inverter (NOT gate):



AND gate:



OR gate:

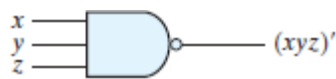


NAND–NOR implementations:

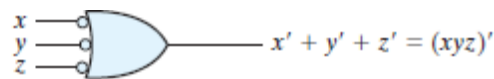
- Digital circuits are frequently constructed with NAND or NOR gates rather than with AND and OR gates.
- NAND and NOR gates are easier to fabricate with electronic components and are the basic gates used in all IC digital logic families.
- Because of the prominence of NAND and NOR gates in the design of digital circuits, rules and procedures have been developed for the conversion from Boolean functions given in terms of AND, OR, and NOT into equivalent NAND and NOR logic diagrams.

Only NAND/NOR gate circuit:

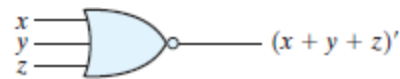
- A convenient way to implement a Boolean function with NAND/NOR gates is to obtain the simplified Boolean function in terms of Boolean operators and then convert the function to NAND/NOR logic.
- The conversion of an algebraic expression from AND, OR, and complement to NAND/NOR can be done by simple circuit manipulation techniques that change AND–OR diagrams to NAND/NOR diagrams.



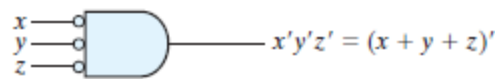
(a) AND-invert



(b) Invert-OR



(a) OR-invert



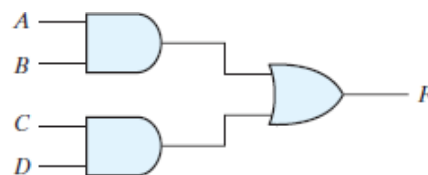
(b) Invert-AND

NAND Implementation Procedure:

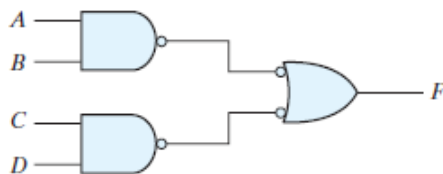
- ✓ Draw the AOI logic of given Boolean expression.
- ✓ Add bubble on input of OR gate & output of AND gate.
- ✓ Add an Inverter on each line that received bubbles.
- ✓ Eliminate double inversions
- ✓ Replace all by NAND gates

Example:

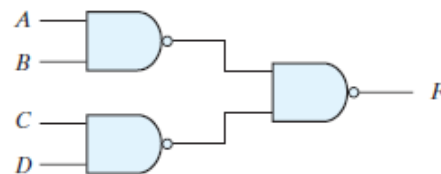
1. Implement $F = AB + CD$ using only NAND gate.



(a)

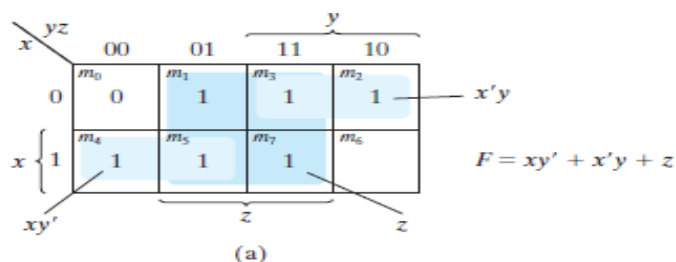


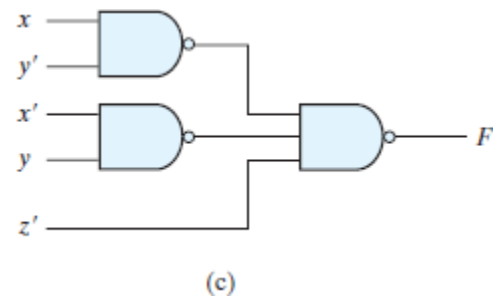
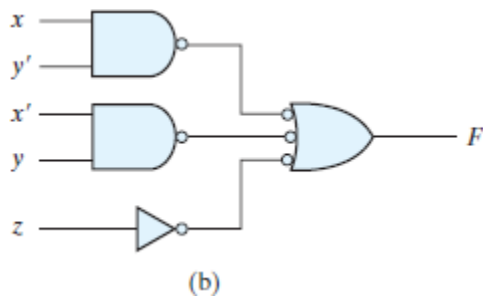
(b)



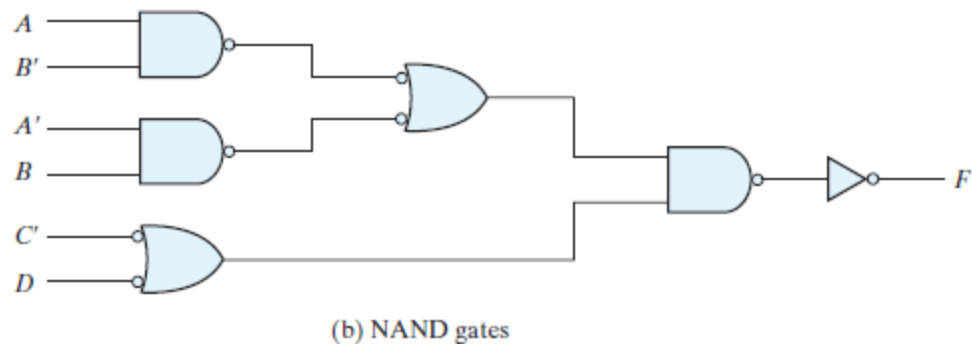
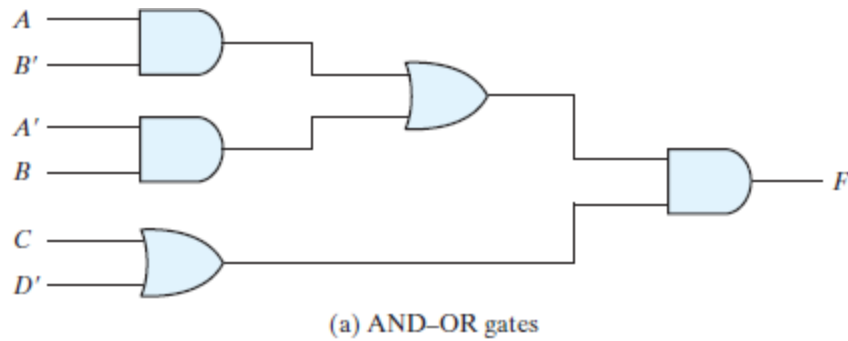
(c)

2. Implement the following Boolean function with NAND gates: $F(x, y, z) = (1, 2, 3, 4, 5, 7)$ (Apr2018)





3. Implement the function $F = (AB' + A'B)(C + D')$ using only NAND gate.

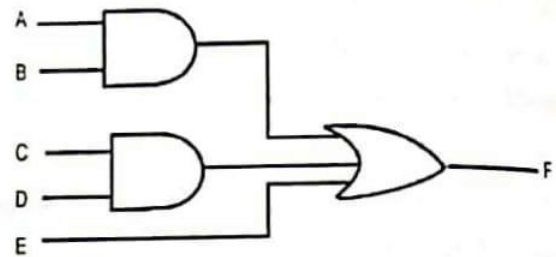


2.7.1 NAND gate implementation

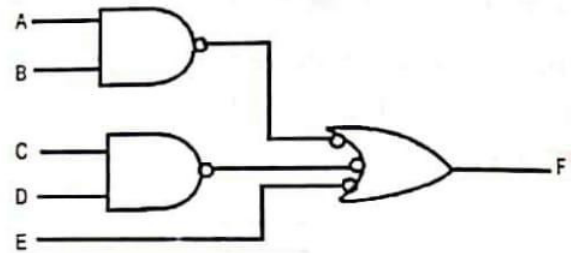
Example 2.10: Implement the Boolean function with NAND gates. $F = AB + CD + E$

Solution:

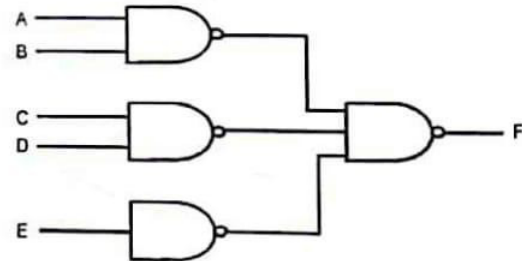
Step 1: Draw AND-OR circuit.



Step 2: Add bubbles on output of each AND gate and input of OR gate.



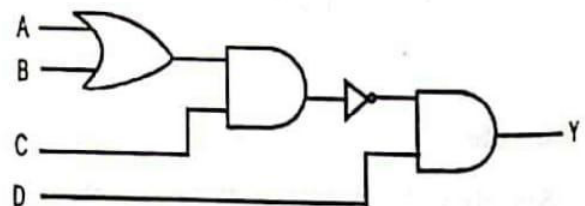
Step 3: Replace other gates by NAND gates.



Example 2.11: Implement the Boolean expression with NAND gates $Y = ((A+B)C)D$

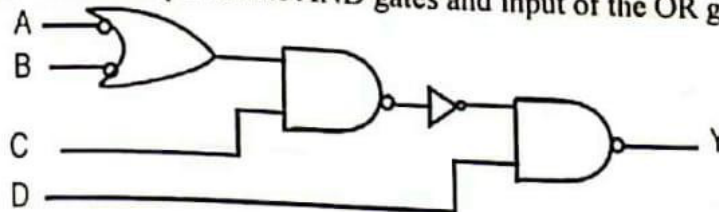
Solution: Step 1: Draw original logic diagram for

$$Y = ((A+B)C)D$$

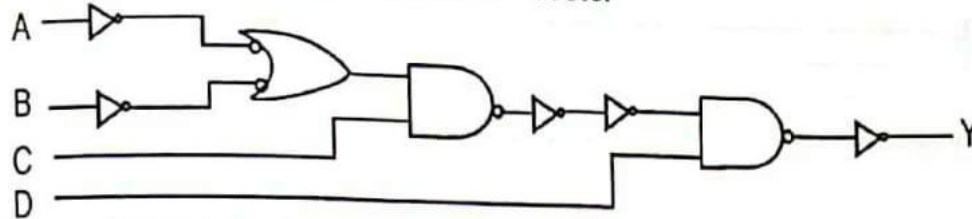


Scanned with
CamScanner

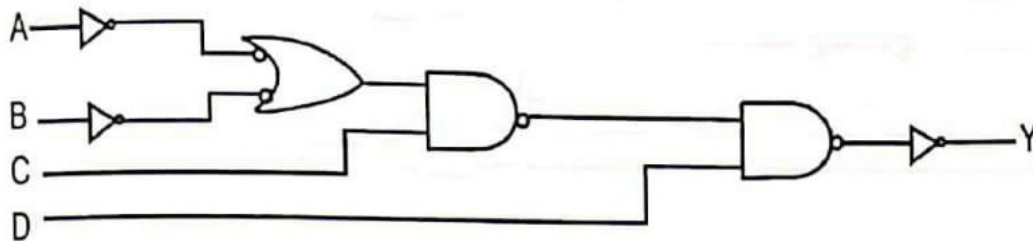
Step 2: Add bubbles on the output of the AND gates and input of the OR gate.



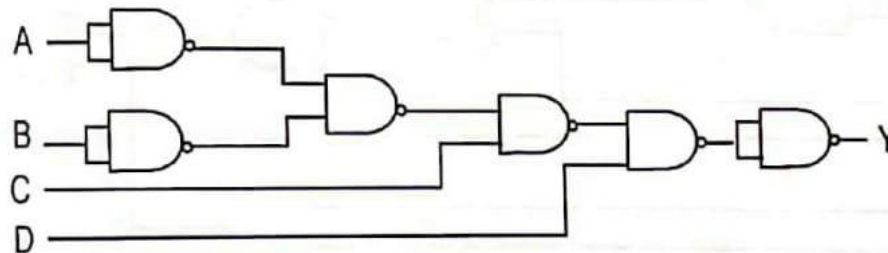
Step 3: Add inverters on each line that received a bubble.



Step 4: Eliminate double inversions.



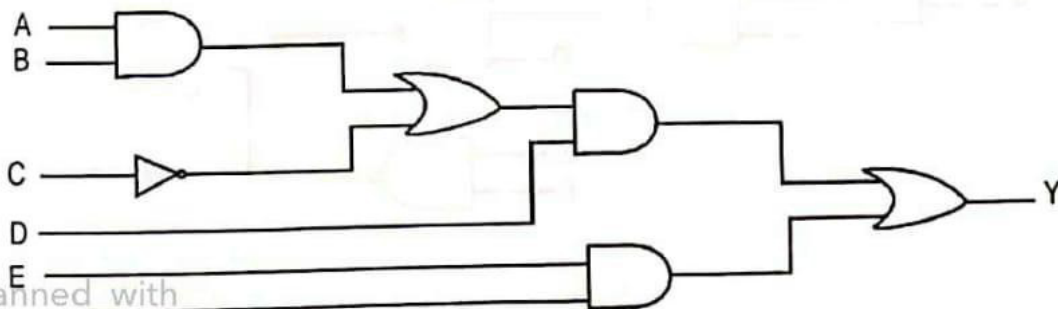
Step 5: Replace the other gates by only NAND gates.



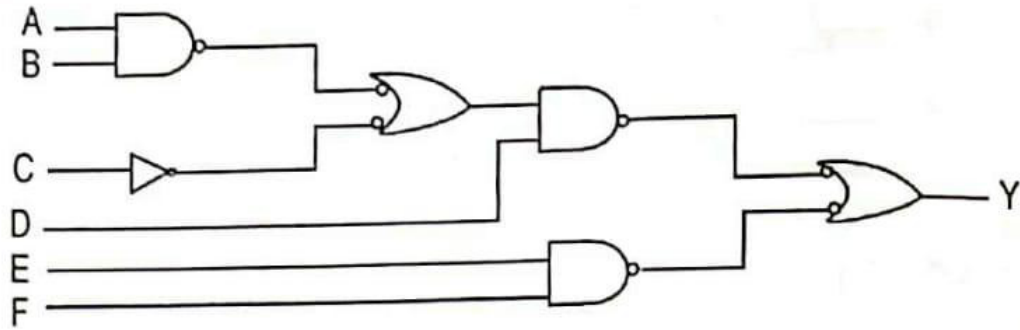
Example 2.12: Implement NAND gates for $Y = (AB + \bar{C})D + EF$.

(April 2004)

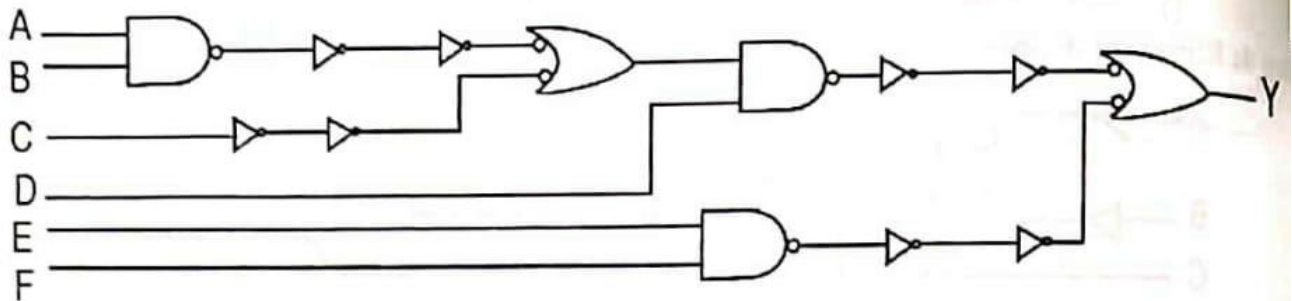
Solution: Step 1: Draw the original logic diagram for the expression $Y = (AB + \bar{C})D + EF$



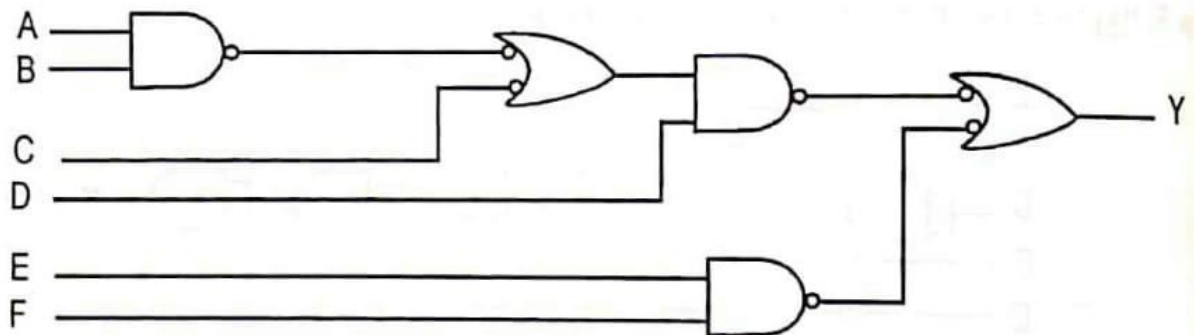
Step 2: Add bubbles on the output of the AND gates and input of the OR gates.



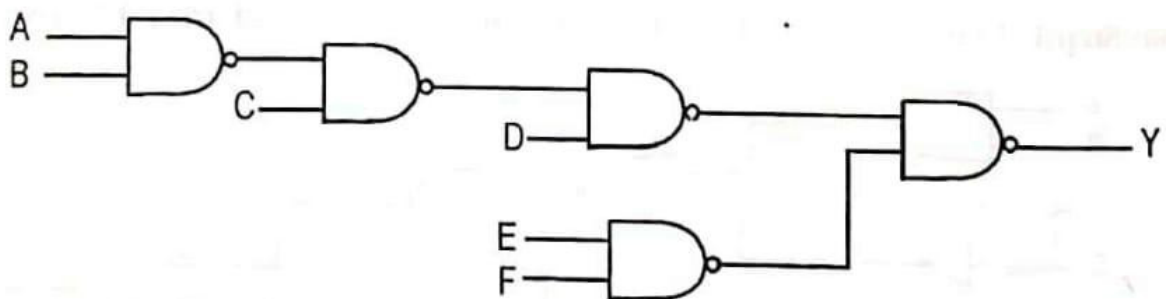
Step 3: Add inverters on each line that received a bubble.



Step 4: Eliminate double inversions.



Step 5: Draw the circuit with only NAND gates.

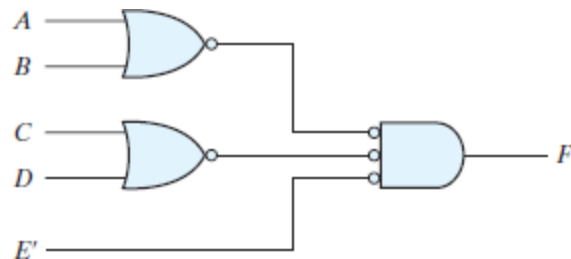


Draw the AOI logic of given Boolean expression.

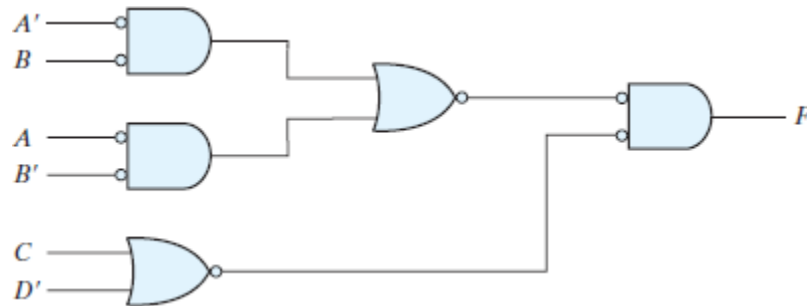
- ✓ Add bubble on input of AND gate & output of OR gate.
- ✓ Add an Inverter on each line that received bubbles.
- ✓ Eliminate double inversions
- ✓ Replace all by NOR gates

Example:

1. Implement $F = (A + B)(C + D)E$ using only NOR gate. (Apr 2018)



2. Implement $F = (AB' + A'B)(C + D')$ using only NOR gate.



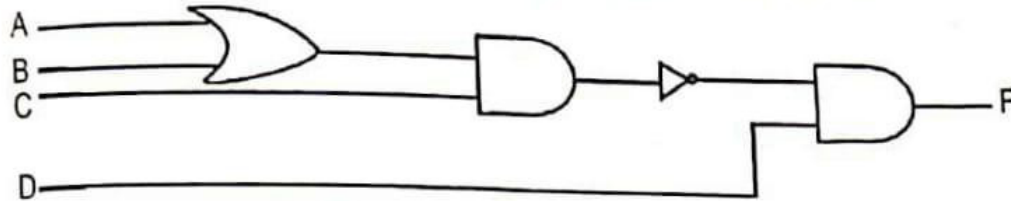
2.7.2 NOR gate implementation

Example 2.13: Implement the Boolean expression with NOR gates.

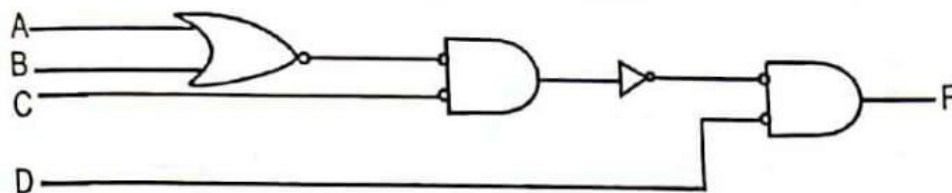
$$F = \overline{(A + B)C}.D$$

Solution:

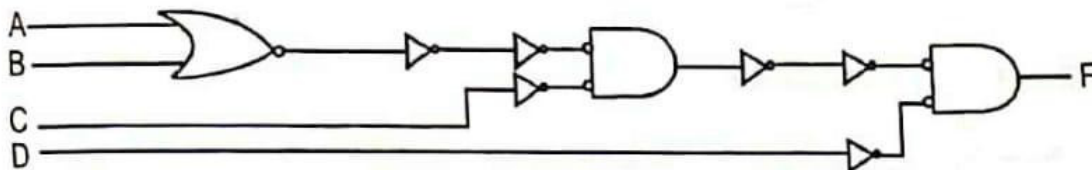
Step 1: Draw the original logic diagram for the given Boolean expression,



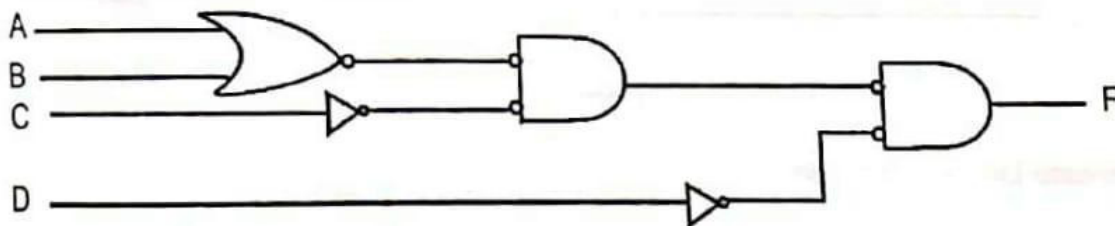
Step 2: Add bubbles on output of each OR gate and add bubbles on input of each AND gate.



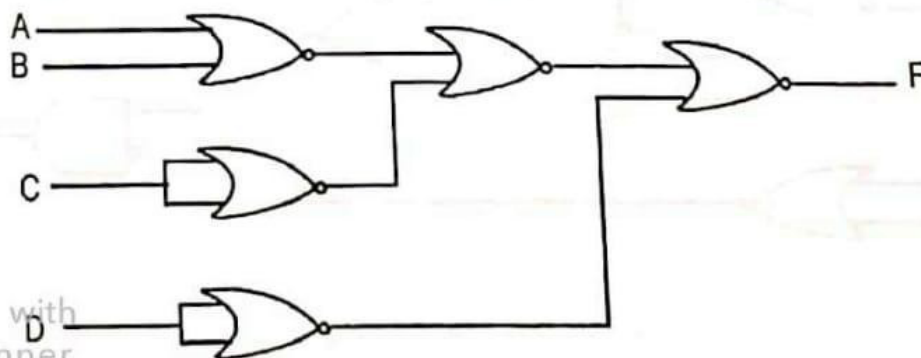
Step 3: Add inverters on each line that recieved bubbles.



Step 4: Eliminate double inversions.



Step 5: Draw the NOR diagram with one graphic symbol.

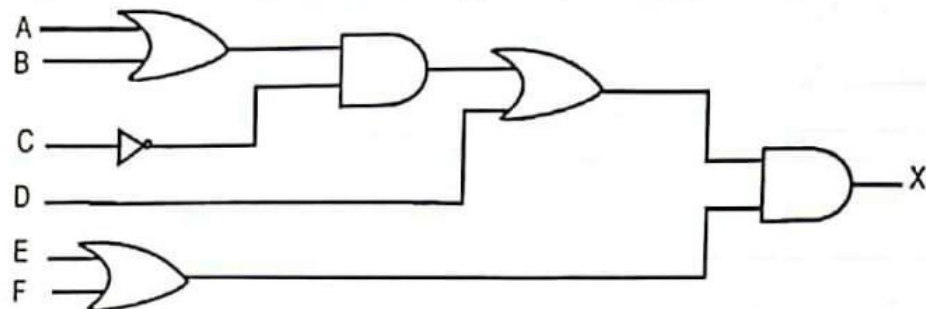


Example 2.14: Draw the multi level NOR circuit for the Boolean expression:

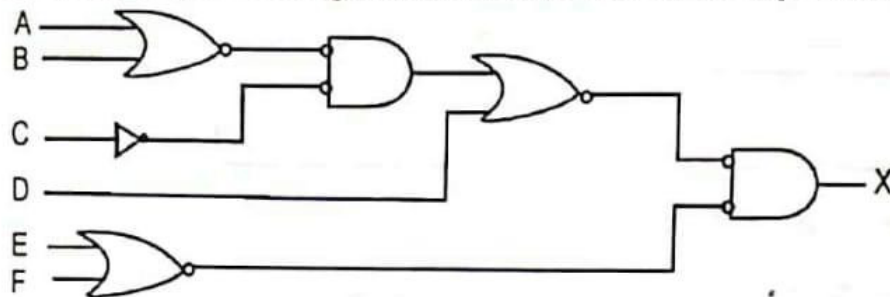
$$X = [(A+B)\bar{C} + D](E+F)$$

Solution:

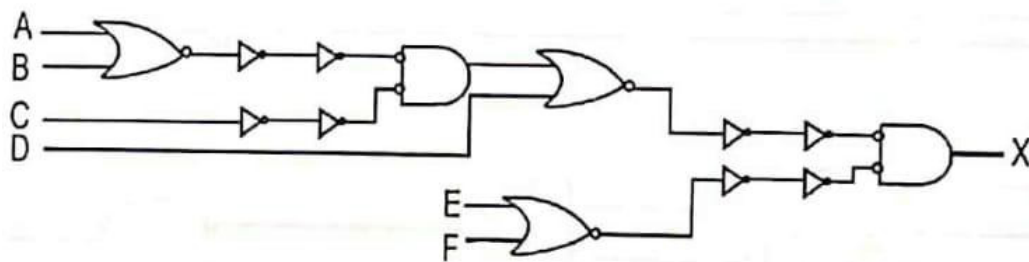
Step 1: Draw the original circuit diagram for $X = [(A+B)\bar{C} + D](E+F)$



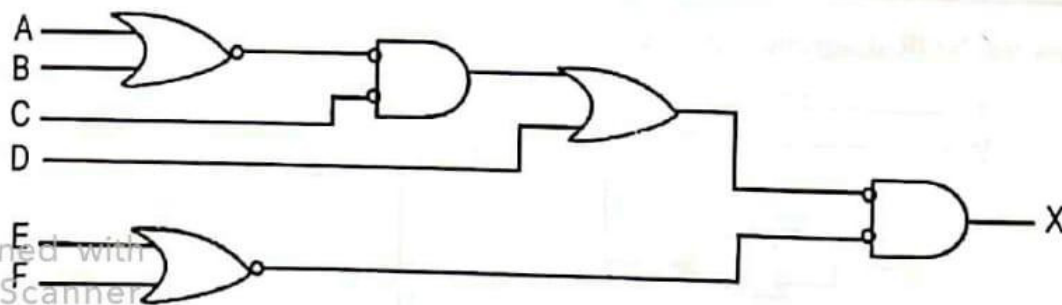
Step 2: Add bubbles on the output of OR gates and add bubbles on the input of AND gates.



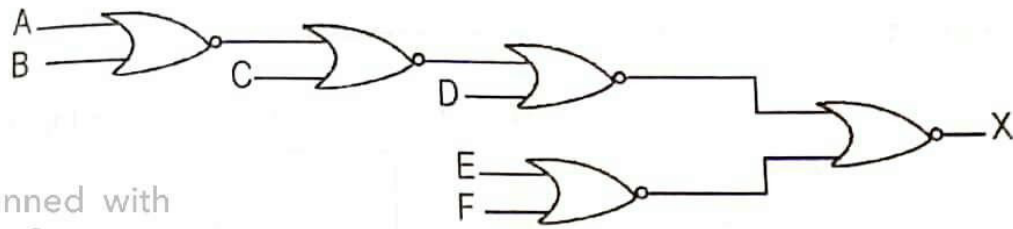
Step 3: Add inverters on each line that received bubbles.



Step 4: Eliminate Double Versions.



Step 5: Draw the NAND diagram using one graphic symbol.



Scanned with
CamScanner